



MODULE :

Le Cloud Computing & DevOps

Pr. F. Benabbou

Master DSBD

2024-2025

Faculté des Sciences Ben M'Sik Casablanca





TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- La Virtualisation
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, ...
- Étude de cas et projet pratique

02 DevOps & Cloud

- La philosophie DeVops
- Version control systems (git)
- **Intégration Continue CI**
- **Déploiement Continu CD**
- **Tests automatisés**
- Infrastructure en tant que Code (IaC)
- Surveillance et Journalisation
- Étude de cas et projet pratique



02

CD/CT





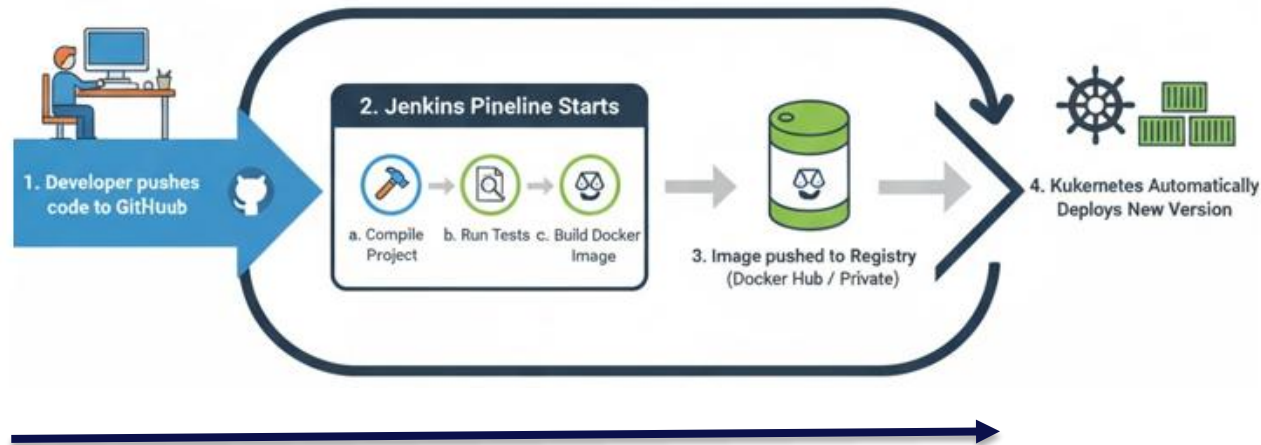
Déploiement des artefacts en DevOps

- Il existe plusieurs méthodes de déploiement qui dépendent de la complexité du projet et de l'infrastructure:
 - Déploiement direct sur un serveur(Ansible)
 - Conteneurisation (Docker) et Orchestration (Docker Swarm, **Kubernetes**)
 - Plateformes Cloud /PaaS
 - ✓ Exemples : Heroku, AWS Elastic Beanstalk, Google App Engine.



Cas d'étude : Kubernetes

- Kubernetes, souvent abrégé k8s, est une plateforme d'orchestration de conteneurs open-source créé par Google en 2015.



Pipeline CI/CD



Kubernetes

- Kubernetes est système qui gère automatiquement des applications conteneurisée.
- Son rôle est d'automatiser le déploiement, mettre à l'échelle (scaling), et orchestrer les conteneurs.





Rôle de Kubernetes

- **Automatisation du Déploiement :**

- ✓ sélection intelligente du nœud où va s'exécuter chaque conteneur
- ✓ allocation des ressources et déploiement sans intervention humaine.

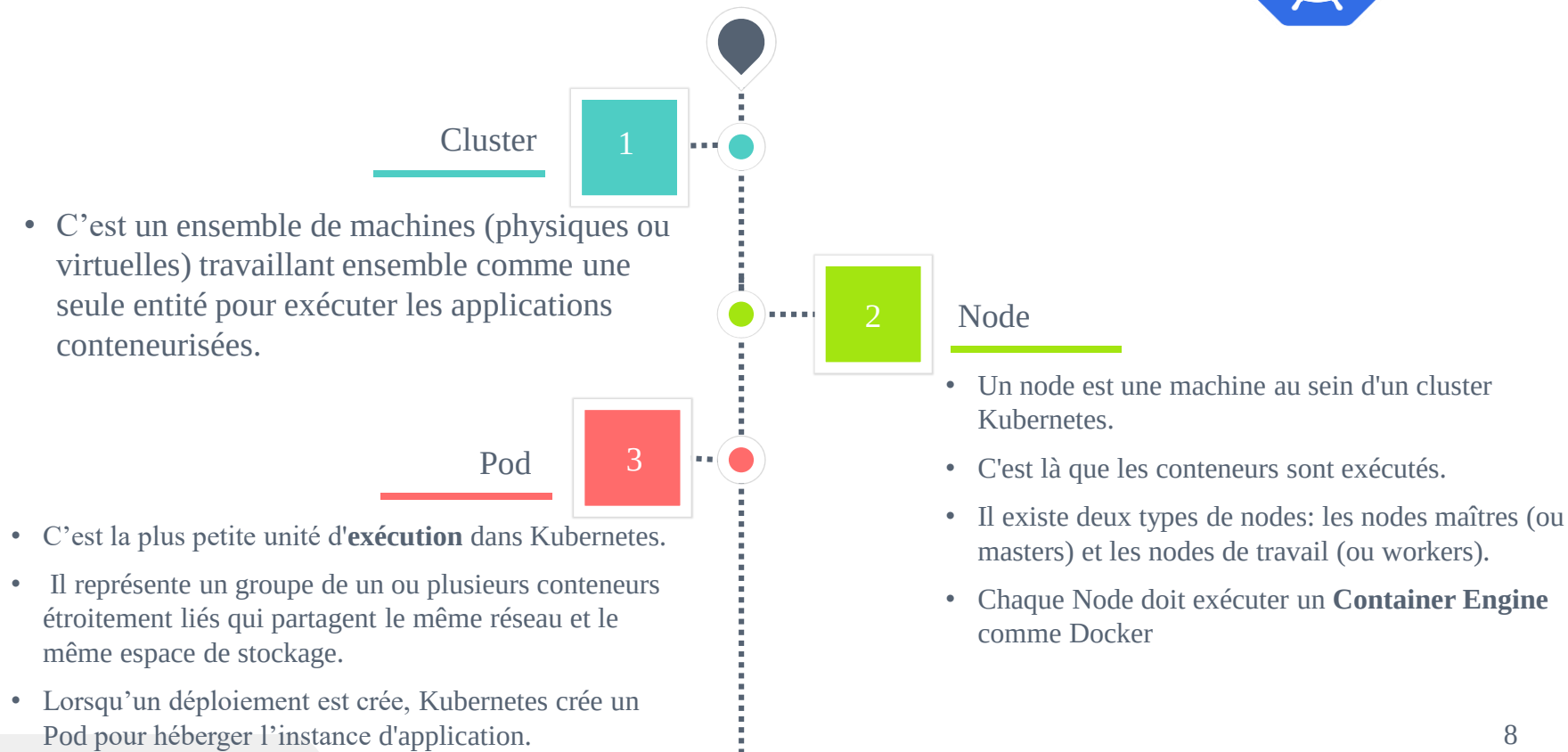
- **Mise à l'échelle automatique (Scaling) :**

- ✓ Kubernetes ajuste le nombre de conteneurs en fonction du trafic utilisateur,
- ✓ il peut répliquer ou supprimer des instances pour maintenir la performance.

- **Gestion des Conteneurs (Self-Healing) :**

- ✓ Gestion proactive les conteneurs,
- ✓ il assure le redémarrage automatique, la découverte de services,
- ✓ et le remplacement des conteneurs en cas de panne.

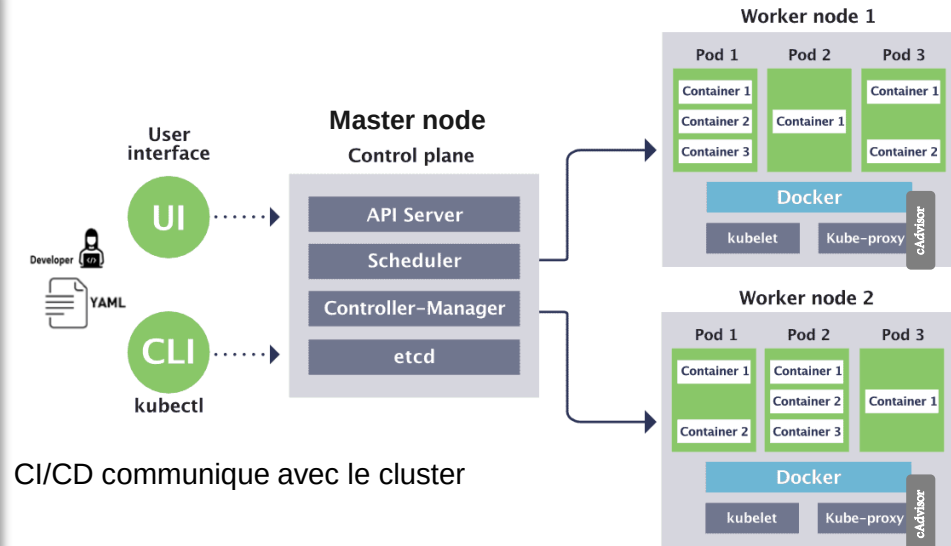
Architectures de base





Master node

- **API server** : Point d'entrée pour **interagir** avec le cluster, il reçoit les commandes (via CLI / YAML) et communiquer avec les autres composantes.
- **Scheduler** : surveille les **ressources** disponibles sur les serveurs et décide sur quel nœud lancer les pods.
- **Controller Manager**: il **maintient** l'état désiré du cluster, compare l'état actuel avec l'état désiré spécifié dans les fichiers de configuration.
- **Etc** : La **base de données** (clé-valeur) qui stocke toute la configuration actuelle du cluster en temps réel



➔ Envoie d'un fichier YAML / une commande CLI (kubect)

UI: interface graphique permettant de visualiser l'état du cluster.

worker node

■ Kubelet :

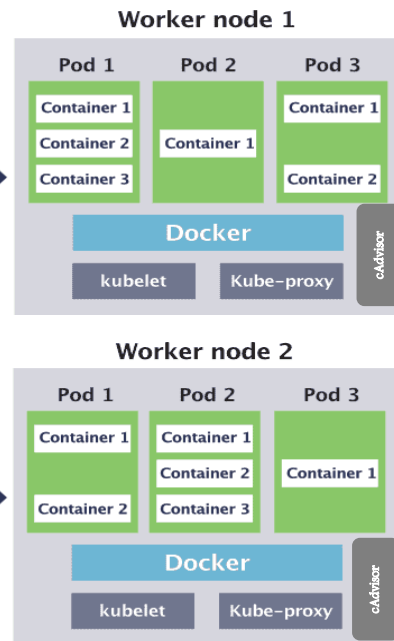
- ✓ c'est un agent qui s'exécute sur chaque worker node.
- ✓ il est responsable de la **gestion des pods** et de leurs conteneurs,
- ✓ il **interagit** avec le **master** et **docker engine** pour la gestion des conteneurs.

■ Kube proxy :

- ✓ gère le routage réseau au sein du cluster,
- ✓ expose les services à l'extérieur du cluster,
- ✓ assure la communication entre les services et les pods,
- ✓ répartit les charge de service entre les différents nodes, etc.

- **cAdvisor** : surveille les performances des conteneurs et collecte des métriques en temps réel sur les conteneurs (CPU, RAM, réseau, etc)

Les workers exécutent les pods



Exemple de commande avec Minikube



- Minikube est un outil qui permet de créer et de gérer un cluster Kubernetes local sur une seule machine.

- `kubectl get pods`
- `kubectl apply -f webapp.yaml`
- `kubectl describe pod <name>`
- `Kubectl get deployments`
- `Kubectl describe services/webapp-service`
- `kubectl delete -f webapp.yaml`



Objet Kubernetes

- Un objet Kubernetes est une entité persistante stockée dans la base de données du cluster (etcd).
- Il spécifie l'état souhaité (Desired State) que Kubernetes essaiera de maintenir.
- Il existe plusieurs objet Kubernetes
 - ✓ **Namespace, Pod, Deployment, Service,**
 - ✓ **ConfigMap, Secret, Ingress,**
 - ✓ **PersistentVolumeClaim (PVC), PersistentVolume (PV),**
 - ✓ **Job, CronJob, StatefulSet, Role / ClusterRole et RoleBinding**

Objets kubernetes

- Chaque objet est défini par un fichier YAML (manifeste), ce dernier doit obligatoirement comporter quatre champs de premier niveau: `apiVersion`, `Kind`, `Metadata`, et `spec` (sauf `Secret` et `ConfigMap`).
 - ✓ **apiVersion** : c'est la version de l'API server de Kubernetes (Pod: v1, Deployment apps/v)
 - ✓ **Kind** spécifie le type d'objet créé, il existe de nombreux objets dans Kubernetes comme Pod, Service, Volume, Namespace, Deployment et d'autres.
 - ✓ **Metadata** est la section qui décrit l'objet:
 - *Name*: le nom de l'objet qu'on veut créer.
 - *Namespace* : L'espace de nom où va l'objet être mis
 - *Labels*: paires clé-valeur qu'on associe à un objet pour l'identifier facilement, et faciliter sa sélection (ex: pour supprimer tous les pods ayant un label spécifique).
 - *Annotations* sont également des paires clé-valeur qui peuvent contenir des informations supplémentaires.
 - ✓ **spec** est la section contenant les configurations techniques de l'objet à créer (ex. pour un pod, il faut spécifier les conteneurs qu'il va exécuter).



Namespace (Namespace.yaml)

- L'objet Namespace dans Kubernetes sert principalement à organiser et isoler les ressources dans un cluster.

```
apiVersion: v1
kind: Namespace
metadata:
  name: calculatrice-app
  labels:
    env: production          #Permet de filtrer les ressources par environnement
    network-policy: Secure  #utilisé pour définir des politiques de sécurité
spec:
  finalizers:               #utilisé pour empêcher la suppression immédiate d'un objet
    - kubernetes            #Kubernetes doit effectuer un nettoyage interne avant supp.
```

Secret (calculatrice-secret.yaml)

- Le Secret permet de stocker les informations sensibles comme les mots de passe, clés, etc.

```
apiVersion: v1                                #Version de l'API Kubernetes pour les secrets
kind: Secret                                   # Type d'objet : stockage sécurisé
metadata:
  name: db-credentials                         # Nom unique du secret dans le namespace
  namespace: calculatrice-app                 # Isolé dans le namespace de l'app
  labels:                                     # permettent de classer et retrouver le secret facilement
    app: calculatrice                         # c'est une étiquette
    env: production                           # Permet de filtrer les ressources par environnement
type: Opaque                                  #le contenu du Secret est un ensemble de paires Clé/Valeur
data:                                         # c'est où sont défini les informations sensibles
  DB_PASSWORD: cGFzc3dvcmQxMjM=              #Le password est codé en base64
```

ConfigMaps (calculatrice-config.yaml)

- L'objet ConfigMap permet de stocker des données de configuration que les pods peuvent utiliser.

```
apiVersion: v1                                #Version de l'API Kubernetes pour les configMap
kind: ConfigMaps                               # Type d'objet : dictionnaire de configuration
metadata:
  name: calculatrice-config                    # Nom de la configMap
  namespace: calculatrice-app                 # Isolé dans le namespace de l'app
  labels:                                     # Les labels permettent de classer et retrouver le secret facilement
    app: calculatrice
    tier: frontend                             # Permet de filtrer les ressources par type
data:                                          # configuration du comportement de l'appli Java sans toucher le code
  JAVA_OPTS: "-Xms256m -Xmx512m"             #Définit la RAM initiale allouée et le max a ne pas dépasser (MV)
  UI_COLOR: "blue"                           #Permet de changer certaine valeur de variables d'env de l'appli.
                                              #l'apparence de l'interface sera modifiée lors du déploiement.
  API_ENDPOINT: http://api.calculatrice.local # Adresse du service avec l'app doit communiquer
```

Pod (calculatrice-pod.yaml)

- Ce n'est pas nécessaire de créer un fichier pod.yaml; On définit généralement le Pod à l'intérieur du Deployment, mais voici sa structure pour exemple.

```
apiVersion: v1                #Version de l'API Kubernetes
kind: Pod                     # Type d'objet : pod
metadata:
  name: calculatrice-pod      # Nom de la configMap
  namespace: calculatrice-app # Isolé dans le namespace de l'app
  labels:                     # Les labels permettent de classer et retrouver le secret facilement
    app: calculatrice
spec:
  containers:                 # Liste des conteneurs qui vont tourner dans ce Pod.
  - name: java-app            # Nom interne du conteneur
    image: mon-repo/calculatrice-java:v1.2 # L'image Docker à télécharger et exécuter
    ports:                    #Déclaration des ports réseau du conteneur.
    - containerPort: 8080     # Le port sur lequel l'application Java écoute réellement.
    envFrom:                  # ajout des clés d'une ConfigMap comme var d'environnement.
    - configMapRef:
      name: calculatrice-config # Référence au nom exact de l'objet ConfigMap
```

Service

- Un Service permet de rendre une application accessible à **l'intérieur** ou à **l'extérieur** du cluster.
- Dès qu'un Service est créé, Kubernetes lui attribue une adresse IP interne appelée **ClusterIP** statique.
 - ✓ donc Service permet à un groupe de Pods d'être joignable même si les pods sont supprimés et remplacés par d'autres avec une autre adresse.
 - ✓ Fait du load balancing en répartissant les requêtes sur les workers.
 - ✓ Permet la communication entre services au sein d'un cluster
- Le Service utilise des **labels** pour trouver les Pods



Service

- Dès qu'un Service est créé, Kubernetes lui attribue une adresse IP interne appelée **ClusterIP** statique.
- Le champ **type** indique comment le service **doit être exposé**
 - ✓ **ClusterIP**: on expose le service avec une adresse **IP stable** qui n'est accessible que depuis **l'intérieur** du cluster.
 - ✓ **Nodeport**: expose, à **l'extérieur** du cluster, un service via un **port** sur chaque node, et pas d'équilibrage de charge.
 - On peut accéder à l'application par : **http://<IP_du_node>:<NodePort>**
 - ✓ **LoadBalancer**: expose un service sur **Internet** avec une **IP publique** unique, il y a équilibrage de charge entre les nodes.

Service (calculatrice-service.yaml)

```
apiVersion: v1
kind: Service                                # Type d'objet : service
metadata:
  name: calculatrice-service                # Nom du service
  namespace: calculatrice-app
spec:
  type: ClusterIP                           # IP interne au cluster, non accessible depuis Internet
  selector:                                # selectionne les pod qui sont concernés
    app: calculatrice                      # Envoie le trafic vers les Pods ayant ce label
  ports:
    - protocol: TCP                        # Protocole réseau utilisé
      port: 80                             # Port sur lequel le Service écoute à l'int. du cluster.
      targetPort: 8080 #C'est le port sur lequel l'appli écoute à l'int. du Pod
```



Deployment

- Un Deployment est un objet Kubernetes qui permet de déployer et gérer automatiquement des applications conteneurisées.
 - ✓ Crée et gère les Pods
 - ✓ Assure le nombre de réplicas (instances)
 - ✓ Met à jour l'application selon un déploiement progressive
 - ✓ Redémarre automatiquement les Pods en cas de problème
- le Déploiement est décrit dans un fichier YAML

Deployment (Calculatrice-Deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment                                # Type d'objet : deployment
metadata:
  name: calculatrice-java                      # Nom du  déploiement
  namespace: calculatrice-app
spec:
  replicas: 3                                  # Nombre d'instances de l'app (pods) à créer et à maintenir en vie
  selector:                                    # définit les critères de sélection des pods gérés par ce déploiement.
    matchLabels:                               # défini le critère de sélection, ce déploiement gère les pods ayant l'étiquette "app: calculatrice".
      app: calculatrice                       # Doit correspondre exactement aux labels du template
  template:                                    # définit le modèle des pods qui seront créés par le déploiement.
    metadata:
      labels:                                  # Étiquettes collées sur chaque Pod créé
        app: calculatrice
    spec:                                       # Spécification technique du conteneur
      containers:
        - name: java-app                      # Nom du conteneur dans le Pod
          image: mon-repo/calculatrice-java:v1 # Spécifie l'image Docker à utiliser pour le conteneur.
          ports:                               # Définit les ports exposés par le conteneur
            - containerPort: 8080             # Port sur lequel l'application Java écoute
          env:                                 # Variables d'environnement injectées dans l'appli Java depuis Secret et ConfigMap
            - name: APP_COLOR
              valueFrom:                       # Nom de la variable dans le code Java
                configMapKeyRef:               # Source de la valeur
                  name: calculatrice-config    # Nom de la ConfigMap
                  key: UI_COLOR                # Clé à extraire
            - name: DB_PASS
              valueFrom:                       # Variable pour le mot de passe
                secretKeyRef:
                  name: db-credentials         # Nom du Secret source
                  key: DB_PASSWORD             # Clé à extraire
```

Objet Ingress

- L'objet Service gère le réseau à **l'intérieur** du cluster, l'Ingress expose les applications au monde réel (production)
- En général son rôle inclus
 - ✓ Le Routage par Nom de Domaine: Il lit l'URL de l'utilisateur et dirige la requête vers le bon service.
 - ✓ Le Routage par Chemin: il peut aussi découper une seule URL en plusieurs blocs pour diriger vers différents services (/app, /images...).
 - ✓ La Gestion de la Sécurité (Certificats SSL/TLS): Au lieu d'installer un certificat de sécurité sur chaque Pod on l'installe une seule fois sur Ingress.



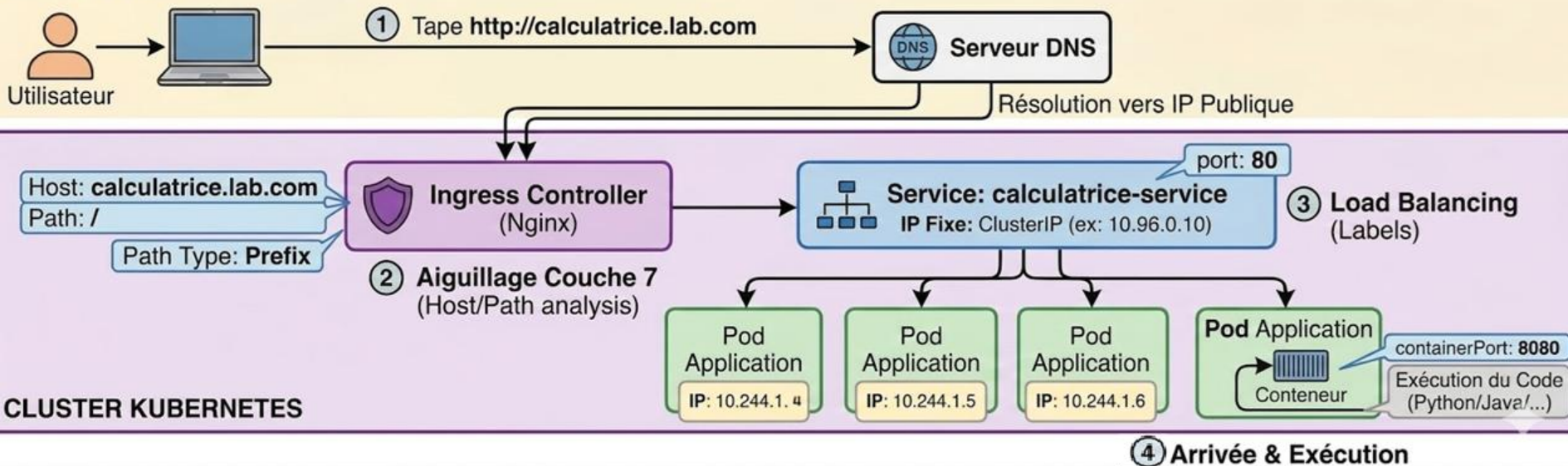
Ingress:(calculatrice-ingress.yaml)

```
apiVersion: networking.k8s.io/v1
kind: Ingress                                # Type d'objet : routage HTTP externe
metadata:
  name: calculatrice-ingress
  namespace: calculatrice-app
  annotations:                                # Application sera gérée par le contrôleur Nginx
    kubernetes.io/ingress.class: nginx
spec:
  rules:                                      # Liste des règles de routage
  - host: calculatrice.lab.com                # Nom de domaine DNS pour l'accès à l'application
    http:
      paths:
      - path: /                               # Toutes les requêtes commençant par /
        pathType: Prefix
        backend:                               # Destination du trafic
          service:
            name: calculatrice-service        # Ingress ne communique pas directement avec les Pods
            port:
              number: 80                       # Port du Service cible
```

Exemple de fonctionnement

■ Exemple avec l'application

MONDE EXTÉRIEUR





Helm Templates

- Ils permettent de générer automatiquement des objets Kubernetes sans trop perdre du temps à les écrire (<https://helm.sh/>).
- Les templates sont des fichiers YAML avec des variables dynamiques.
- Helm regroupe dans des packages appelés Charts, tous les fichiers YAML (Deployment, Service, Ingress...) paramétrés et prêts à être déployés,
- Avantages
 - ✓ Réutilisation du code
 - ✓ Réduction des erreurs
 - ✓ Déployer la même app sur plusieurs environnements (dev, staging, prod)

Fonctionnement : values.yaml

- Les informations de configuration qui personnalisent un chart sont mis dans un fichier values.yaml.
- Ce fichier contient toutes les valeurs utilisées par les templates
- Le but est de séparer logique (template) et la configuration (values)
- Il permet de personnaliser le déploiement sans modifier le template
- Facilite l'adaptation à plusieurs environnements (dev, test, prod)
- le Chart.yaml contient les métadonnées du chart lui-même.

```
calculatrice-chart/  
├─ Chart.yaml  
├─ values.yaml  
└─ templates/  
    ├─ deployment.yaml  
    ├─ service.yaml  
    ├─ configmap.yaml  
    ├─ secret.yaml  
    └─ ingress.yaml
```

Helm chart template : exemple

- Nous allons transformer calculatrice-deployment.yaml en une template
- On va remplacer les noms et les valeurs par des balises de variables.

Template deployment.yaml

original

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deploy
  namespace: {{ .Values.namespace }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Values.appName }}
  template:
    metadata:
      labels:
        app: {{ .Values.appName }}
    spec:
      containers:
        - name: java-app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.targetPort }}
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: calculatrice-java
  namespace: calculatrice-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: calculatrice
  template:
    metadata:
      labels:
        app: calculatrice
    spec:
      containers:
        - name: java-app
          image: mon-repo/calculatrice-java:v1
          ports:
            - containerPort: 8080
```

Helm chart template : exemple

- Il suffit d'utiliser un fichier values.yalm pour récupérer les données.

Template deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deploy
  namespace: {{ .Values.namespace }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Values.appName }}
  template:
    metadata:
      labels:
        app: {{ .Values.appName }}
    spec:
      containers:
        - name: java-app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.targetPort }}
```

values.yaml

```
# Informations sur l'application
appName: calculatrice-java
namespace: calculatrice-app
replicaCount: 3
# config image Docker
image: repository: mon-repo/calculatrice-j
tag: "v1"
# Configuration réseau
service:
  targetPort: 8080 # Le port interne de to
application Java
  port: 80 # Le port exposé par le
Service Kubernetes
```



Avantages de Kubernetes

- **Isolation:** Les **namespaces** permettent d'isoler les applications et les équipes.
- **Scalabilité:** Les pods peuvent être créés et détruits dynamiquement pour s'adapter à la charge.
- **Disponibilité:** Kubernetes assure la haute disponibilité des applications en redémarrant automatiquement les pods défaillants.
- **Flexibilité:** Kubernetes permet de déployer des applications complexes et hétérogènes.



L'écosystème Kubernetes

- Kubernetes est un ensemble de standards.
- Il existe de nombreuses variantes (ou "flavours") de Kubernetes, qui définissent des solutions techniques pour les fonctionnalités de Kubernetes.
- Exemple:
 - Google Kubernetes Engine (GKE) (Google Cloud Platform).
 - Azure Kubernetes Services (AKS) (Microsoft Azure).
 - Elastic Kubernetes Services (EKS) (Amazon Web Services).